interpolation_check.py

James Evans

07/19/2025

def check_prime()

```
# Input: points are array of points, values are array of values where each point evaluates, j is index of points
# Output: true if point j has the tangent plane lays below all the points.
def check_prime(points, values, j):
    N, d = points.shape
    g = cp.Variable(d)   # the subgradient varaible at point j

    constraints = []
    for i in range(N):
        if i == j:
            continue
        lhs = values[i] - values[j]
        rhs = (points[i] - points[j]) @ g
        constraints.append(lhs >= rhs)

    # Use dummy objective since we only care about feasibility
    prob = cp.Problem(cp.Minimize(0), constraints)
    prob.solve()

    return prob.status in [cp.OPTIMAL, cp.OPTIMAL_INACCURATE]

```

We examine the following two lines of code from the program:

```
N, d = points.shape
g = cp.Variable(d)   # the subgradient variable at point j
```

Lines 5-8

Extracting Dimensions

$$N, d = \texttt{points.shape}$$

Here, `points` is a NumPy array of shape (N, d), representing a collection of N points in $\mathbb{R}^d$. That is:

- N is the number of data points.
- d is the dimension of each point.

For example, if we have:

$$\texttt{points} = \begin{bmatrix} 1.0 & 2.0 \\ 3.0 & 4.0 \\ 5.0 & 6.0 \end{bmatrix},$$

then $N = 3$ and $d = 2$, meaning there are 3 points in $\mathbb{R}^2$.

Defining the Subgradient Variable

```
g = cp.Variable(d)
```

This line defines a decision variable $g \in \mathbb{R}^d$ using the `cvxpy` optimization library. The variable g represents a *subgradient* of a convex function at the point $x_j \in \mathbb{R}^d$. We are not taking the gradient of the function itself. Instead, we are checking whether a function *could* be convex by testing if there exists a subgradient at each point consistent with convexity.

Mathematical Context

In convex analysis, a function $f : \mathbb{R}^d \to \mathbb{R}$ is convex if for any point x_j, there exists a subgradient $g_j \in \mathbb{R}^d$ such that the following subgradient inequality holds for all $i \neq j$:

$$f(x_i) \geq f(x_j) + \langle g_j, x_i - x_j \rangle.$$

The program seeks to verify whether such a subgradient g_j exists for each point x_j. The variable `g` is introduced to model this subgradient and is used to formulate the linear constraints required for the feasibility check.

Lines 18-22

We examine the following lines of code from the function `check_prime`:

```
# Use dummy objective since we only care about feasibility
prob = cp.Problem(cp.Minimize(0), constraints)
prob.solve()

return prob.status in [cp.OPTIMAL, cp.OPTIMAL_INACCURATE]
```

This block constructs and solves a convex optimization problem to verify whether the subgradient inequalities at a given point x_j are feasible. The check is based entirely on the existence of a solution, not the optimization of any meaningful objective.

First, the line `cp.Minimize(0)` defines a constant objective function equal to zero. Since we are only interested in whether the constraints can be satisfied, we use a dummy objective to turn the problem into a pure feasibility check.

Next, `prob = cp.Problem(cp.Minimize(0), constraints)` constructs the convex program. The constraints encode the subgradient condition that must hold for a function to be convex at point x_j:

$$f(x_i) \geq f(x_j) + \langle g, x_i - x_j \rangle \quad \text{for all } i \neq j.$$

The command `prob.solve()` invokes the solver. If it finds a feasible vector $g \in \mathbb{R}^d$ satisfying all the inequalities, then the function values are consistent with convexity at x_j.

Finally, the line `return prob.status in [cp.OPTIMAL, cp.OPTIMAL_INACCURATE]` checks whether the solver successfully found such a subgradient. If the solver returns `OPTIMAL`, then the constraints were satisfied exactly. If it returns `OPTIMAL_INACCURATE`, then the constraints were satisfied approximately, which is still acceptable in practice. The function returns `True` in either case, indicating that convexity is locally satisfied at x_j.

def prime_interpolation_check()

```
25  # check primal LP each point j. Use this version of check if we have relative small number of points.
26  # Input: points are array of points, values are array of values where each point evaluates.
27  # Output: true if points can interpolate the convex function.
28  def prime_interpolation_check(points, values):
29      return all(check_prime(points, values, j) for j in range(len(points)))
30
```

Explanation of `prime_interpolation_check`

The function `prime_interpolation_check(points, values)` verifies whether a set of given points in $\mathbb{R}^d$, along with associated scalar values, can be interpolated by a convex function. It does so by invoking a pointwise convexity check based on the subgradient condition.

Inputs

- `points`: an $N \times d$ array representing N points $x_1, \ldots, x_N \in \mathbb{R}^d$.
- `values`: a length-N array of real numbers $f_1, \ldots, f_N$, where $f_i = f(x_i)$ for some unknown function f.

Method

The function checks each index $j = 0, \ldots, N-1$ by calling the helper function `check_prime(points, values, j)`, which attempts to find a subgradient $g_j \in \mathbb{R}^d$ at point x_j such that the subgradient inequality holds for all $i \neq j$:

$$f(x_i) \geq f(x_j) + \langle g_j, x_i - x_j \rangle.$$

In other words, it tests whether the tangent plane at point x_j lies below all other evaluated points. If such a subgradient exists for each j, then the data are consistent with the values being generated by a convex function.

Implementation

The logic is implemented as follows:

```
def prime_interpolation_check(points, values):
    return all(check_prime(points, values, j) for j in range(len(points)))
```

This line iterates through all indices j and ensures that the subgradient test passes at each point. The function returns `True` if and only if the condition holds for every $j \in \{0, \ldots, N-1\}$.

Use Case

This version of the interpolation check is appropriate when the number of sample points N is relatively small, since it solves one linear program per point in the primal space of dimension d.

check_dual()

```python
# Input: points are array of points, values are array of values where each point evaluates, j is index of points
# Output: true if point j has the tangent plane lays below all the points.
def check_dual(points, values, j):
    N = len(points)
    alpha = cp.Variable(N)

    constraints = [
        alpha >= 0,
        cp.sum(cp.multiply(alpha[:, None], points - points[j]), axis=0) == 0
    ]

    objective = cp.Minimize(cp.sum(cp.multiply(alpha, values - values[j])))

    prob = cp.Problem(objective, constraints)
    prob.solve()

    # If the optimal value is -\infty, dual is feasible, then the primal is infeasible
    return prob.status != cp.UNBOUNDED  # return True means primal is feasible
```

Dual Linear Program for Convexity Interpolation

We consider the function `check_dual(points, values, j)`, which tests whether a given set of function values is consistent with convexity at point x_j using a dual linear program (LP). This method provides a certificate for the existence (or failure) of a valid subgradient at x_j.

Problem Setting

Given:

- A set of points $x_1, x_2, \ldots, x_N \in \mathbb{R}^d$
- Corresponding scalar values $f_1, f_2, \ldots, f_N \in \mathbb{R}$

Our goal is to determine whether these values can be interpolated by a convex function. That is, we ask whether there exists a convex function f such that $f(x_i) = f_i$ for all i.

Subgradient Inequality (Primal Condition)

A necessary condition for convexity at point x_j is the existence of a subgradient $g_j \in \mathbb{R}^d$ such that:

$$f(x_i) \geq f(x_j) + \langle g_j, x_i - x_j \rangle \quad \text{for all } i \neq j.$$

This condition forms the basis of the *primal* LP. If such a subgradient exists, then a tangent plane at x_j lies below all other points.

Dual Linear Program

Rather than directly solving for g_j, the dual LP attempts to show whether the primal is infeasible. It introduces dual variables $\alpha \in \mathbb{R}^N$, with the following formulation:

Variables

$$\alpha_1, \ldots, \alpha_N \in \mathbb{R}$$

Constraints

$$\alpha_i \geq 0 \quad \text{for all } i = 1, \ldots, N$$

$$\sum_{i=1}^{N} \alpha_i (x_i - x_j) = 0$$

The first condition ensures non-negativity of the dual weights. The second condition enforces that the weighted displacement from point x_j has zero net direction, i.e., the convex combination of vectors $x_i - x_j$ is centered.

Objective

$$\min_{\alpha} \sum_{i=1}^{N} \alpha_i (f_i - f_j)$$

This can be rewritten as:

$$\sum_{i=1}^{N} \alpha_i f_i - f_j \sum_{i=1}^{N} \alpha_i$$

The objective seeks to minimize the gap between the convex combination of function values f_i and the anchor value f_j.

Interpretation

If the dual LP is *unbounded below*, then the objective can be made arbitrarily negative under the given constraints. In this case, no subgradient g_j can satisfy the subgradient inequality, meaning the primal LP is infeasible and convexity fails at x_j.

On the other hand, if the dual LP has a bounded solution (i.e., the solver status is not `UNBOUNDED`), then the subgradient inequality may hold, and convexity at x_j cannot be ruled out.

Return Value

The code returns `True` if:

$$\texttt{prob.status} \neq \texttt{cp.UNBOUNDED}$$

This indicates that the primal LP is feasible — a subgradient satisfying the convexity condition exists at x_j.

Note: Xuan's implementation swaps the roles of i and j, so the notation in his code does not match the indexing used in my other Polymath Jr write ups.

```
# check dual LP each point j. Use this version of check if we have large number of points.
# Input: points are array of points, values are array of values where each point evaluates.
# Output: true if points can interpolate the convex function.
def dual_interpolation_check(points, values):
    return all(check_dual(points, values, j) for j in range(len(points)))

```